SYSC4001 – Operating Systems – Assignment 1

Brian Quang

#101297103

October 11, 2026

Experimental Report

A. Scheduling Tradeoffs

Commands:

```
./scheduler -i report_scenarios/scheduling.csv -a fcfs -q 1 -c 1 -o
report_output/fcfs_run

./scheduler -i report_scenarios/scheduling.csv -a srtf -q 1 -c 1 -o
report_output/srtf_run

./scheduler -i report_scenarios/scheduling.csv -a rr -q 1 -c 1 -o
report_output/rr_q1_run

./scheduler -i report_scenarios/scheduling.csv -a rr -q 20 -c 1 -o
report_output/rr_q20_run
```

Results

Algorithm	Quantum	Makespan	Avg Response Time	Avg Turnaround Time	Long Process Turnaround
FCFS	-	39	15.00	23.75	21
SRTF	-	40	3.00	16.50	40
RR	1	70	3.75	42.75	70
RR	20	39	15.00	23.75	21

1. **How does changing the RR quantum affect response time and time spent context switching?**

With RR quantum 1, average response time improves to 3.75 because each process gets CPU time sooner. However, frequent switching increases overhead, with 35 time units spent on context switching and a makespan of 70. With quantum 20, RR behaves more like FCFS because processes can run longer before being interrupted. The makespan decreases to 39, while average response time increases to 15.00. Therefore, a smaller quantum improves responsiveness but causes more context-switching overhead.

2. **How does SRTF compare with FCFS in average turnaround time? What happens to the long process?**
   Use at least one specific interval from timeline.csv to support your explanation. Explain the results you obtained, even if they differ from what you expected.

SRTF reduces average turnaround time from 23.75 under FCFS to 16.50 because shorter processes finish sooner. However, the long process (PID 1) has a turnaround time of 40 under SRTF, compared with 21 under FCFS. This happens because shorter jobs interrupt the long process when they arrive. According to timeline.csv, shorter jobs arrive at times 5, 8, and 12, preempting PID 1 before it finishes. Therefore, SRTF benefits short processes but can significantly delay long ones.

CPU and I/O Overlap

Commands:

```
./scheduler -i report_scenarios/baseline.csv -a fcfs -q 1 -c 1 -o
report_output/baseline_run

./scheduler -i report_scenarios/overlap.csv -a fcfs -q 1 -c 1 -o
report_output/overlap_run
```

Workload	CPU Busy	Idle Time	Makespan	CPU Utilization
Baseline	10	3	15	66.67%
Overlap	14	0	14	82.35%

1. When does the second process use CPU time that would otherwise be idle? Give one specific interval from timeline.csv.

In the baseline run, the CPU becomes idle while the first process waits for I/O. In the overlap run, the second process uses this time to execute, reducing wasted CPU time. For example, the interval when the first process is waiting for I/O and the second process is running demonstrates this overlap. This improves CPU utilization because useful work continues while another process is blocked.

2. Why can CPU utilization improve without reducing the makespan?

CPU utilization improves because the second process performs work while the first process waits for I/O. CPU busy time increases from 10 to 14 units, raising utilization from 66.67% to 82.35%. The additional work overlaps with the I/O wait rather than requiring the CPU to remain idle. Therefore, more CPU work is completed efficiently without proportionally increasing the total completion time.